

Methods with Return type in java.

- * A return statement causes the program control to transfer back to the caller of a method. A return type may be a primitive type like int, float, double. a reference type or void type (returns nothing).
- * The type of data returned by a method must be compatible with the return type specified by method, for instance, if the return type of some method is boolean, we can not return an integer.
- * The variable receiving the value returned by a method must also be compatible with the return type specified for method.
- * The parameter can be passed in a sequence & they must be accepted by the method in some sequence.

ex.

```
public class A {  
    int add () { // without Arguments  
        int x = 30;  
        int y = 70;  
        int z = x + y;  
        return z;  
    }  
  
    public static void main (String [] args) {  
        A a1 = new A ();  
        int add = a1.add ();  
        System.out.println ("sum of x & y = " + add);  
    }  
}
```

8/8 => The sum of x & y = 100

ex. 2

```
public class B {
```

```
    int add (int x, int y) // with arguments
```

```
    {
        int z = x + y;
```

```
        return z;
    }
```

```
public void main (String[] args) {
```

```
    B b1 = new B();
```

```
    int add = b1.add (10, 20);
```

```
    System.out.println ("sum of x & y = " + add);
}
```

O/P $\Rightarrow$ sum of x & $y = 30$

* Constructor *

* Purpose! - The main purpose of constructor is initialization of instance variable.

- * Imp points!
- ① Constructor name & class name must be same.
 - ② Constructor is a method only which does not have any return type including void.
 - ③ For each object constructor will be executed only once.
 - ④ Constructor will be executed automatically when we create an object of class.
 - ⑤ Two types of constructor,
 - ① default (no-arg)
 - ② parameterized (args)

Syntax! -

```
class className {
```

```
    className(input args) {
```

```
        // initialization of instance variable
```

```
    }
```

```
}
```

Notes! - ① we can have public, private, protected & default constructor in java.

② In java constructor can not be a final, abstract & static.

③ If there is ^{no} constructor in a class compiler automatically create a default constructor.

* Difference between constructor & methods.

Constructors

Methods

① Constructors name & class name must be same

Method name can be anything

② Constructors is not having any return type include void.

Method is having return type.

③ for each object constructor will executed only once.

for each object method will execute any number of times.

④ Constructors are used to initialize non-static variables.

To write business logic we can use method.

⑤ Constructor will execute automatically when we create object.

we have to call method to execute

⑥ Types (A) Default (B) Parameterized

⑥ Types, (A) static (B) Non-static

ClassName	Obj. Ref. variable	=	new	ClassName();
↓	↓		↓	
User-define datatype	Reference variable,		new is keyword	
Referenced datatype	It will hold reference of object.		To create new object in heap memory.	
Non-primitive datatype	It will hold memory address of an object		create instance variable	
			It will return memory address of an object	

ClassName();

↓
It is a constructor.

use to initialize instance variable

There are two types of constructor

① default.

② parameterized.

using assignment operator.
that memory address
will be stored inside
a object.

* Use of this keyword →

In java this is a reference variable that refers to the current object. this is a keyword. this is a reference variable to current class object instance members. we can use this keyword any number of times.

Use →

① To access current class instance members.

② Instance members :- instance variables & methods (instance)

Note → We can use this keyword anywhere except static method & static block.

* this is a keyword.

* this is a reference variable.

* this is a keyword used to refer current class instance members (instance variables & method)

* this keyword we can use to differentiate between local and global variable

* we can use this keyword inside a method & inside a constructor.

```
ex. class Test {  
    int i;  
    void setValue (int i) {  
        this.i;  
    }  
    void show() {  
        System.out.println(i)  
    }  
}
```

```
class xyz {  
    public static void main(String[])  
    {  
        Test t = new Test();  
        t.setValue(10);  
        t.show();  
    }  
}
```

O/P → 10

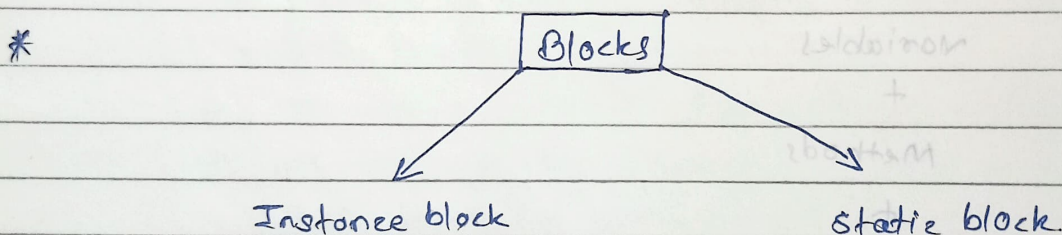
```

    {
        System.out.println("Instance block execution");
    }

    {
        public static void main(String[] args) {
            System.out.println("main method execution");
            FinalDemoBlock f1 = new FinalDemoBlock();
        }
    }
}

```

O/P ⇒ static method execution
 main method execution
 Instance method execution
 constructor method execution



Def. <pre> Class ClassName { { // instance block // access static & instance variable } } </pre>	<pre> Class ClassName { static { // static block // we can access only static variable. } } </pre>
--	--

execution. when we create object of class before constructors instance block will execute	At the time of class file loading in JVM memory static block will execute.
---	--

Use of blocks → we can write time consuming code inside a blocks. like JDBC connectivity code

Instance block

static block

① Instance block deal with object
static block deals with ~~object~~ class

② { // instance block
}
static { // static block
// we can access only static variable.
}
}

execution. when we create object of class before constructors instance block will execute.
At the time of .class file loading in JVM memory static block will execute.

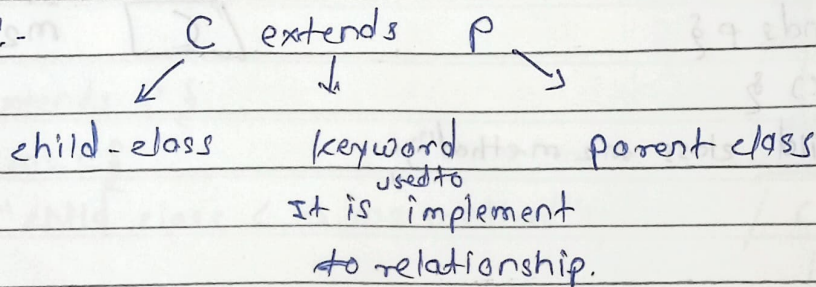
③ Instance block can access static & non-static variables.
static block can access only static variables.

④ execute at the time of object creation, before constructor execution.
execute at the time loading .class file in JVM memory.

* Inheritance

- * The process ~~at~~ where one class acquires the properties (variables & methods) of another class.
- * The process of acquire parent class properties in child class.
- * The process in which one object can acquires all properties & methods of a parent class.

* Syntax :-



* Types of Inheritance:

- ① Single or simple
- ② multi-level
- ③ Hierarchical
- ④ multiple
- ⑤ Hybrid
- ⑥ cyclic

Note - ① Java supports for single, multilevel & Hierarchical inheritance by using classes & interfaces.

② Java supports for multiple, Hybrid inheritance by using interface.

* Single / simple Inheritance:

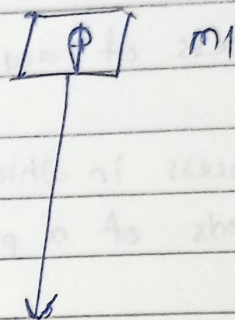
→ single parent & single child

```
class P {  
    void m1() {  
        println("parent class m1 method");  
    }  
}
```

```
class C extends P {  
    void m2() {  
        println("child class m2 method");  
    }  
}
```

```
P p1 = new P();  
p1.m1();
```

```
C c1 = new C();  
c1.m1();  
c1.m2();
```



5 sentence Inheritance,

- ① The process where one class acquire properties of the another class.
- ② The process of acquire parent class properties in child class.
- ③ The process in which one object can acquire all properties & methods of parent class.
- ④ Java support for single, multilevel, hierarchical inheritance by using class & methods.

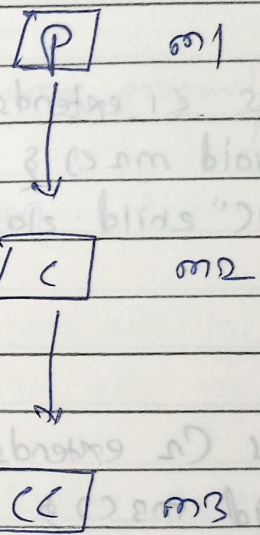
* Multi-level Inheritance

→ single parent & single child but in multiple level

```
Class P {  
    void m1 () {  
        println ("parent class P → m1 method");  
    }  
}
```

```
Class C extends P {  
    void m2 () {  
        println ("child class C → m2 method");  
    }  
}
```

```
Class CC extends C {  
    void m3 () {  
        println ("child class CC → m3 method");  
    }  
}
```



```
P p = new P();  
p.m1();
```

```
C c = new C();  
c.m1();  
c.m2();
```

```
CC cc = new CC();  
cc.m1(); cc.m3();  
cc.m2();
```

⑥ Java support multiple, Hybrid inheritance by using interfaces

* Hierarchical Inheritance!

→ single parent & multiple childs.

```
Class P {
```

```
void m1() {
```

```
    println("parent class P → m1 method");
```

```
}
```

```
Class C1 extends P {
```

```
void m2() {
```

```
    println("child class C1 → m2 method");
```

```
}
```

```
Class C2 extends P {
```

```
void m3() {
```

```
    println("child class C2 → m3 method");
```

```
}
```

```
P p = new P();
```

```
p.m1();
```

```
C1 c1 = new C1();
```

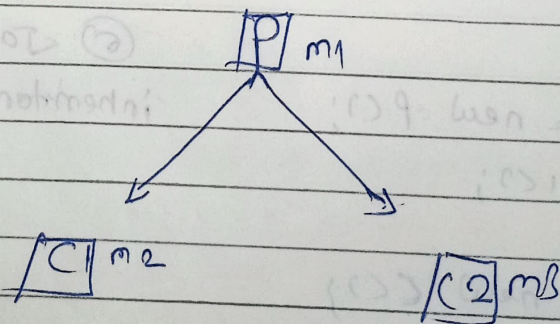
```
c1.m2();
```

```
c1.m1();
```

```
C2 c2 = new C2();
```

```
c2.m3();
```

```
c2.m1();
```



Super keyword & Super call & This call

- * It is a non-static final reference variable,
- * Super keyword holds super-class (parent) class information.
- * From sub-class if we want to refer super class instance members then use super keyword.
- * To differentiate subclass instance variables or instance methods from superclass instance variables & instance methods.

* Note: we can not use super keyword in static method & static method blocks.

* this() & super();

* this() & super() both are constructor calls, to call super class constructor & current class constructor.

* this() & super() both we can use in constructor only as a first line only.

* this() & super() we can use only one at a time but not both simultaneously.

```
public class AAA {  
    int x = 666;  
    void m1 (int x) {  
        syso (this.x);  
    }  
}  
  
public class BBB extends AAA {  
    int x = 777;  
    void m2 (int x) {  
        syso (x);  
        syso (this.x);  
        syso (super.x);  
        super.m1 (10);  
    }  
}  
  
public static void main (String [] args) {  
    new BBB (). m2 (888);  
}
```

ex ②
 public class CCC {
 CCC() {
 sysout.println("only CCC constructor");
 }

~~CCC()~~
 CCC(int x) {
 this();
 sysout.println("int only CCC constructor");
 }
 }

public class DDD extends CCC {
 DDD() {
 super(); / super(10);
 }

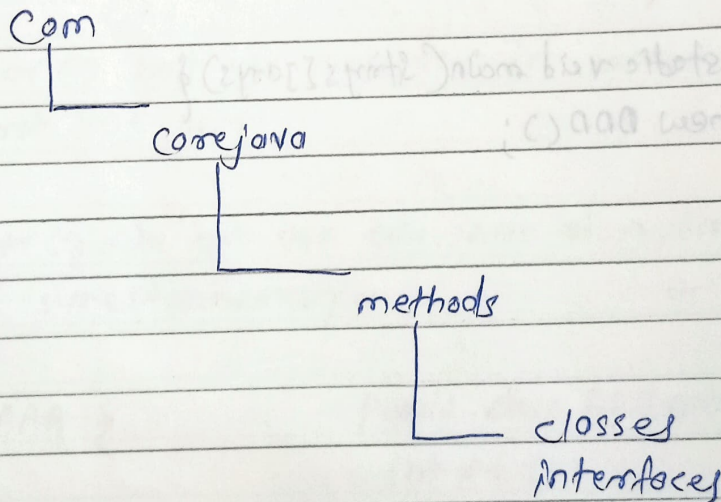
public static void main(String[] args) {
 new DDD();
 }
 }

* Package & Imports:-

- * Package is a simple folder.
- * Package is a encapsulation mechanism to group related classes & interface into a single unit called as package.
- * In real time every class & interface is a part of package only.
- * In java program we can write at most one package statement. Package stat. is first statement in your java programme.

* Package Name.

Package: com. corejava. methods



* Import statements:

P1
package P1;
class Test {
}

P2
Package P2;
import P1. Test
Class Demo {
 ps v main() {
 var Test = new Test();
 }
}

Test. Java

1st statement	Package statement	→ 0 or 1
2nd statement	Import statement	→ 0 or any no. of times
	Class declaration	→ 0 or multiple times
	Interface declaration	
	main	→ 0/1/ or multiple main

* Access modifiers in Java:

- ① Private
- ② <default> or <package> - private
- ③ Protected
- ④ Public

* Note: ① Private, protected & public are the keywords.
 ② default is not keyword it is just concept.

* Use: → Access modifier will tell us "where to access" class variables & methods.

→ To set access permissions we can use Access modifier.

* Public (Global level)

* Access: within the package + outside the package

* Default: (Package level)

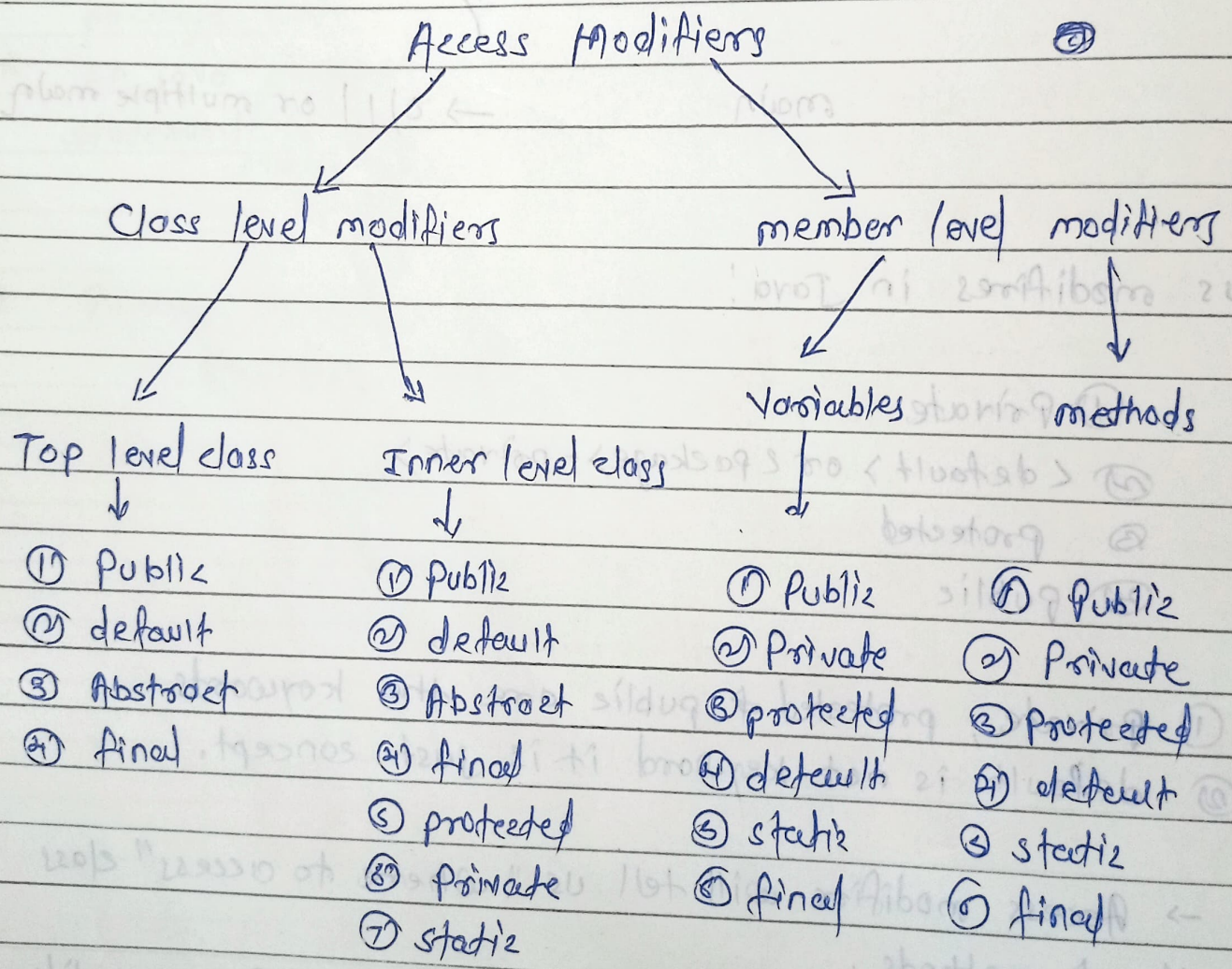
* Access: within the package only.

* Private (class level)

Access -> within the class only.

* Protected :

Access -> within the package + outside of package only in child classes using child references.



Encapsulation.

```
package com.jbk.eneap;  
public class Student {
```

```
    private String name;  
    private double marks;  
    private int rollNum;
```

```
    public Student(String name, double marks, int rollNum) {
```

```
        this.name = name;  
        this.marks = marks;  
        this.rollNum = rollNum;  
    }
```

```
    public String getName() {  
        return name;  
    }
```

```
    public void setName(String name) {  
        this.name = name;  
    }
```

```
    public double getMarks() {  
        return marks;  
    }
```

```
    public void setMarks(double marks) {  
        set this.marks = marks;  
    }
```

```
    public int getRollNum() {  
        return rollNum;  
    }
```

```
    public void setRollNum(int rollNum) {  
        this.rollNum = rollNum;  
    }
```

```

public class main {
    public static void main (String[] args) {

```

```

        student s = new student ("Joy", 99, 202);

```

```

        s.setName ("Joy Baba");

```

```

        s.setMarks (99.5);

```

```

        s.setRollNum (12);

```

```

        String name = s.getName();

```

```

        double marks = s.getMarks();

```

```

        int rollNum = s.getRollNum();

```

```

        syso (name);

```

```

        syso (marks);

```

```

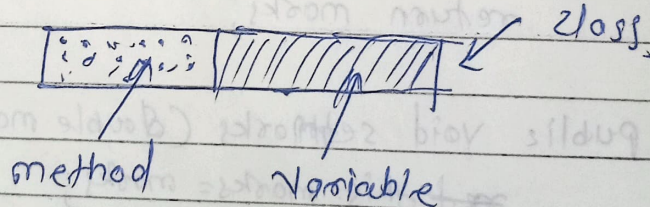
        syso (rollNum);
    }
}

```

Encapsulation

* It is a grouping mechanism where we can group variables & methods into a single unit called as class.

→ class is a best example of encapsulation.



Purpose → Hide data for security purpose

* How to achieve encapsulation?

- (1) By making variables (data) as private.
- (2) By making method as public.

eg. `Class JbKBank {`
`private long accountNumber;`
`private double balance;` → Data hiding -- by making variable as private.

```
public void getBalance() {  
    if (validateUser) {  
        // authenticated user  
        sopln(balance);  
    } else {  
        // unauthenticated user  
        sopln("please provide valid username");  
    }  
}
```

* Hiding a data behind a method called as encapsulation.

5 points on class.

- ① class is a template or blueprint from which objects are created.
- ② class is group of object which have common properties & methods.
- ③ class is used to define user defined datatype.
- ④ class does not require memory for execution.
- ⑤ class is a logical entity.
- ⑥ class determines how objects behave which object will contain.
- ⑦ All objects given in class which have some property & methods.
- ⑧ class is keyword which is used to define class.

⑨

Object

- ① Object is an instance of a class.
- ② we can not create object without class.
- ③ Object is collection of data & methods.
- ④ object is an entity which has states & behavior.

Polymorphism

many forms.

→ The process of providing multiple implementation to a method to perform one operation based on subtype or based on parameter type is called as polymorphism.

Two types of polymorphism

Compile time
(Static polymorphism)

Run time (handle by JVM)
(Dynamic polymorphism)

Method
Overloading

Method
Overriding (ex)

Class Test

void show (inta)

```
{
    println("1");
}
```

void show (int a, int b)

```
{
    println("2");
}
```

P.S.V.M {

```
Test t = new Test();
t.show(10, 20);
```

O/P ⇒ 2

- ① Same Name
- ② same class
- ③ Different Arg.
 - No. of arg.
 - seq. of arg.
 - types of arg.

- ① same name
 - ② diff class
 - ③ same arg.
 - No. of arg.
 - Seq. of arg.
 - type of arg.
- IS A Relation

① Can we overload main method?

→ Yes, we can have any no. of main methods in a class by method overloading. This is because JVM always calls main method which receives String array arg. only.

② Method Overriding

ex. class Test

```
{  
    void show()  
    {  
        sop("1");  
    }  
}
```

class xyz extends Test

```
{  
    void show()  
    {  
        sop("2");  
    }  
}
```

P. S. V. m §

```
Test t = new Test();  
t.show();
```

O/P ⇒ 1

```
XYZ x = new XYZ();  
x.show();
```

O/P ⇒ 2

① Overriding method must have same return type or subtype?

→ From Java 5.0 onwards it is possible to have different return type for a overriding method in child class, but child's return type should be sub-type of parent's return type. is called covariant return type.

* Method Overloading!

If the class is having multiple methods with same name but different argument list.

- * IMP points!:
- ① Two or more methods is having same name
 - ② methods must be defined in same class.
 - ③ methods with different argument list.
 - number of arg. can be different
 - sequence of arg. can be different
 - type of arg. can be different

Note:- Overloaded methods may have the same or diff. return type.

* Constructor Overloading!

— If the class is having multiple constructors with same name but diff. arg. list.

- * IMP points!:
- ① Two or more constructors is having same name
 - ② Constructors must be defined in same class.
 - ③ Constructors with diff. arg. list.
 - number of arg. can be diff
 - sequence of arg. can be diff
 - type of arg. ———

* Method Overriding:-
→ If child class has the same method as declared in the parent class it is called method overriding.
→ Re-defining parent class method in the child class called as method overriding.

* Imp points:-

- ① method must have the same name as in the parent class
- ② method must have the same parameters as in the parent class.
- ③ There must be IS-A relationship.
- ④ method overriding occurs in two classes that having parent-child relationship.

* Notes:-

- ① Both parent class & child class methods must have the same method name.
- ② Both parent class & child class methods must have the same return type.
- ③ Both parent class & child class methods must have the same parameter list.
- ④ we can not override the method declared as final & static.

Overloading

Overriding

① If the class is having multiple methods with the same name but diff. arg. list

① Redefining the parent class method in child class.

② method overloading is performed in same class

method overriding occurs in two classes that having parent-child relationship.

- ③ In method overloading parameter list must be different
- ③ In method overriding parameter list must same
- ④ method overloading is a example of compile time polymorphism
- ④ method overriding is a example of run time polymorphism
- ⑤ In method overloading methods may have the same or diff. return type
- ⑤ In method overriding both parent class & child class methods must have same return type
-

Method Overriding

- ① Instance method
- ② In method overriding method resolution is based on run time object
- ③ Runtime polymorphism
- ④ method resolution will be at runtime.

Method hiding

- ① static method & global variable

- ② In method hiding method resolution is based on type

- ③ Compile time polymorphism

- ④ method resolution will be at compile time
-

5 points on class.

- ① class is template or blueprint from which objects are created.
- ② class is a group of objects which have common properties & method.
- ③ class determines how objects behave & which object will contains.
- ④ class is keyword which is used to define class.
- ⑤ class is a logical entity.
- ⑥ class is used to define userdefine datatype.
- ⑦ class does not require memory for execution.
- ⑧ Inside a class we can create constructor, methods & block etc.

Object.

- ① object is a collection of data & methods
- ② object is an instance of class.
- ③ object is real world entity.
- ④ object is an entity which has states & behave.
- ⑤ objects having require memory for execution.
- ⑥ we can not create object without class
- ⑦ we can create many objects of one class.
- ⑧ we can access class by creating an object.

encapsulation.

- ① It is a group mechanism where we can ^{create} group variables & method into a single unit as called class.
- ② class is a best example of encapsulation.
- ③ Hiding the data behind a methods is called encapsulation.
- ④ we can achieve encapsulation by making variables as ~~as~~ private.
- ⑤ we can achieve encapsulation by making methods as a public.
- ⑥ Data hiding means ~~the~~ making of variables private.

obj. refer.

- ① obj. is a reference variable.
- ② It will hold reference of obj.
- ③ To access functionality of an object.
- ④ To access member of obj.